

PHASE 3 DOCUMENT

EduGenie: Google Gemini Powered Learning Assistant

Phase Name: Google Gemini API Integration and Core Backend Development

Project Name	EduGenie: Google Gemini Powered Learning Assistant
Phase	Phase 3 – Gemini API and Backend Development
Project Type	AI-Powered Learning Application
Main Technology	Google Gemini API
Prepared By	_____
Guide / Faculty	_____
Date	_____

Academic Project Phase Documentation

1. Phase Overview

This is Phase 3 of **EduGenie: Google Gemini Powered Learning Assistant**. This phase implements the secure server-side connection between EduGenie and Google Gemini. It focuses on backend setup, protected API-key handling, authenticated AI requests, prompt construction, response processing, error management, and the first working version of the AI learning-assistant service.

2. Phase Objective

The objective of this phase is to enable EduGenie to receive a student learning request, securely send a structured prompt to Google Gemini through the backend, receive an AI-generated response, process it safely, and return it to the authenticated student interface. The design ensures that the Gemini API key is never exposed to the browser or stored in public source code.

3. Backend Setup

A backend application is created using a suitable framework such as Node.js with Express or Python with FastAPI/Django. The backend is responsible for receiving requests from the EduGenie frontend, checking authentication and permissions, validating inputs, communicating with Google Gemini, formatting outputs, and returning responses. The backend may also connect to a database for users, sessions, saved learning content, and usage records.

4. Secure API-Key Management

The Gemini API key must be stored in a protected environment variable or secure secrets-management service. It should not be written directly in frontend JavaScript, HTML files, mobile application code, screenshots, shared documents, or public GitHub repositories. Local development may use a protected environment file that is excluded from version control. Production deployment should use the hosting platform's encrypted environment or secrets configuration.

5. Core API Endpoint Design

EduGenie can provide a protected backend endpoint such as **POST /api/ai/ask**. The endpoint receives a validated request containing the selected learning mode, subject or topic, and the student question. The backend confirms that the user is logged in, limits invalid or excessive requests, creates a controlled prompt, sends it to Google Gemini, handles the result, and returns a structured response. Other endpoints may support quiz generation, study planning, summaries, and project assistance.

6. Request Validation

Before calling the AI service, the backend checks whether the required fields are present and whether the request has an allowed learning mode. It removes or limits unnecessarily long input where needed, rejects malformed requests, and checks the authenticated user session. Validation helps protect the service from incorrect input, excessive traffic, and misuse. The application should use clear messages such as "Please enter a question" or "Your session has expired; please log in again."

7. Prompt Construction

The backend converts the user request into a structured prompt that tells Gemini how to respond. For example, when the student chooses an explanation mode, the prompt can request a clear academic explanation with examples and simple language. When the student chooses quiz mode, the prompt

can request a specified number of questions, answers, and short explanations. Prompt templates should keep the response relevant to learning and should avoid passing sensitive or unnecessary personal information to the AI service.

8. Gemini Request and Response Flow

1. A logged-in student sends a request from the EduGenie interface.
2. The frontend sends the request to the protected backend endpoint.
3. The backend verifies authentication and validates the input.
4. The backend selects the appropriate prompt template.
5. The backend sends the generated prompt to Google Gemini using the protected API key.
6. Gemini returns generated content to the backend.
7. The backend checks the response, handles errors, and formats the content.
8. The frontend receives and displays the response to the student.
9. If enabled, the user can save the response in learning history.

9. Response Processing

The backend should handle the Gemini response carefully before returning it to the frontend. It can extract the relevant generated text, remove unsupported formatting where necessary, limit excessively long responses, and attach useful notices such as “Verify important information with your faculty or textbook.” The application should present a graceful fallback if no usable response is received.

10. Error Handling

The service should handle invalid user input, authentication failure, network errors, unavailable Gemini service, API quota limits, server errors, and empty responses. The user interface should show understandable feedback instead of technical stack traces. For example: “The AI service is temporarily unavailable. Please try again shortly.” Detailed error logs may be retained securely for developer troubleshooting without exposing sensitive information to users.

11. Security Controls

Security controls include HTTPS for data transmission, authenticated frontend-to-backend requests, role-based access checks, server-side API-key storage, input validation, output encoding, request-rate limits, restricted cross-origin configuration, and audit logs for important administrative actions. The system should not store raw conversation data longer than required, and access to saved learning history should be limited to the appropriate user or authorized administrator.

12. Initial Feature Implementation

The initial implementation for this phase can provide one main feature: **Ask EduGenie**. A student selects a topic or enters a question, and the backend sends an explanation-oriented prompt to Gemini. The returned response is displayed in the student chat interface. This working feature confirms that the secure integration path is successful and creates the foundation for later modules such as quiz generation, study planning, and project assistance.

13. Testing Plan

Testing in this phase includes environment-variable checks, API connectivity tests, valid and invalid request tests, login-protected endpoint tests, prompt-template tests, response-format tests,

error-condition tests, request-rate checks, and confirmation that the API key is not visible in frontend files or public repositories. Test users should try short questions, long questions, empty input, unsupported requests, and simulated service failures.

14. Phase Deliverables

The deliverables are a configured backend service, protected Gemini API-key setup, authenticated AI endpoint, input-validation rules, prompt templates for the initial learning mode, Gemini request and response handling, error messages, security controls, initial Ask EduGenie feature, and test evidence for successful integration.

15. Planned Next Phase

The next phase will be **Phase 4: AI Learning Assistant Interface and Core Learning Features**. It will focus on the student-facing user interface, chat experience, concept explanation controls, response saving, session management, and the integration of the completed backend service into the EduGenie dashboard.

16. Conclusion

Phase 3 establishes the secure technical connection that powers EduGenie. By placing Gemini communication in the backend, protecting the API key, validating user requests, and handling responses responsibly, the project gains a reliable base for building student-facing AI learning features in the next phase.

Phase Approval

Prepared by	_____
Guide / Faculty	_____
Approved by	_____